

DBMON-03: NoSQL Database Monitoring

Series: DBMON — Database Monitoring | **Notebook:** 3 of 7 | **Created:** March 2026 |
Last Updated: 09/18/2026

Overview

This notebook covers monitoring NoSQL databases with Dynatrace, including document stores (MongoDB, Cosmos DB), key-value stores (DynamoDB), and wide-column stores (Cassandra). You will learn how to analyze operation performance, read/write ratios, collection-level metrics, and replica health using distributed trace spans.

Table of Contents

- [1. NoSQL Database Overview](#)
 - [2. MongoDB Monitoring](#)
 - [3. DynamoDB Monitoring](#)
 - [4. Cassandra Monitoring](#)
 - [5. Cosmos DB Monitoring](#)
 - [6. Read vs Write Analysis](#)
 - [7. Cross-Database Comparison](#)
 - [8. Summary and Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
OneAgent	Deployed on application hosts with NoSQL database clients
Permissions	<code>storage:spans:read</code> , <code>storage:entities:read</code>
Data	Application traffic generating NoSQL database calls
Prior Reading	DBMON-01: Database Monitoring Fundamentals

1. NoSQL Database Overview

NoSQL databases use different data models and operations than SQL databases. Dynatrace captures these differences through span attributes:

Database	Category	<code>db.system</code>	Common Operations	Key Characteristics
MongoDB	Document Store	<code>mongodb</code>	<code>find</code> , <code>insert</code> , <code>update</code> , <code>delete</code> , <code>aggregate</code>	BSON documents, replica sets
DynamoDB	Key-Value / Document	<code>dynamodb</code>	<code>GetItem</code> , <code>PutItem</code> , <code>Query</code> , <code>Scan</code> , <code>BatchWriteItem</code>	Partition keys, provisioned/on-demand capacity
Cassandra	Wide-Column	<code>cassandra</code>	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> (CQL)	Partition keys, consistent hashing
Cosmos DB	Multi-Model	<code>cosmosdb</code>	<code>ReadItem</code> , <code>CreateItem</code> , <code>Query</code> , <code>ReplaceItem</code>	Request Units (RU), multiple consistency levels
Couchbase	Document Store	<code>couchbase</code>	<code>get</code> , <code>upsert</code> , <code>query</code> , <code>search</code>	Memory-first, N1QL query language

Let's discover which NoSQL databases are active in your environment.

In community practice the thresholds in this notebook are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.

```
// Field names corrected 08/12/2026 – pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
// db.operation -> db.operation.name (stable; set on 50,379 of
57,295 db spans)
// db.statement -> db.query.text (stable; 33,463)
// db.mongodb.collection-> db.collection.name (stable; 6,912)
// db.name -> db.namespace (stable; 57,281)
// Confirm the catalog for your tenant with:
// fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Discover active NoSQL databases
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
"couchbase", "hbase"})
// db.operation.name is empty on some instrumentations – fall back to
code.function (see §2)
| fieldsAdd operation = coalesce(db.operation.name, code.function)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    unique_operations = countDistinct(operation)
}, by:{db.system, server.address}
| sort call_count desc
```

2. MongoDB Monitoring

MongoDB is the most widely used document database. Dynatrace captures MongoDB operations through the official MongoDB driver instrumentation, providing visibility into query patterns, collection access, and replica set behavior.

Key MongoDB Span Attributes

Attribute	Description	Example
<code>db.system</code>	Always <code>mongodb</code>	<code>mongodb</code>
<code>db.operation.name</code>	MongoDB command	<code>find</code> , <code>insert</code> , <code>aggregate</code>
<code>db.namespace</code>	Database name	<code>orders</code> , <code>inventory</code>
<code>db.collection.name</code>	Target collection	<code>users</code> , <code>products</code>
<code>server.address</code>	MongoDB host (or mongos for sharded)	<code>mongo-primary.internal</code>

Which field carries the command depends on the instrumentation — check `db.operation.name` first, then `code.function` (OneAgent MongoDB) or `span.name` (OTel Redis). The MongoDB queries below fall back to `code.function` when `db.operation.name` is empty. Note that `code.function` is `experimental` in the semantic dictionary (read 09/18/2026), so re-check it before building long-lived dashboards on it.

```
// MongoDB operation breakdown by collection
// db.operation.name is the dictionary field, but OneAgent's MongoDB capture
// may leave it null
// and carry the command in code.function – fall back so the breakdown is not
// silently empty.
fetch spans, from:-1h
| filter db.system == "mongodb"
| fieldsAdd operation = coalesce(db.operation.name, code.function)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by:{db.namespace, db.collection.name, operation}
| sort call_count desc
| limit 25
```

```
// MongoDB slow operations – find operations exceeding 100ms
fetch spans, from:-1h
| filter db.system == "mongodb"
| filter duration > 100ms
| fields start_time, db.namespace, db.collection.name, db.operation.name,
    db.query.text, duration_ms = duration / 1ms
| sort duration_ms desc
| limit 20
```

```
// MongoDB operation volume over time
// Falls back to code.function where the instrumentation leaves
db.operation.name empty (see §2).
fetch spans, from:-6h
| filter db.system == "mongodb"
| fieldsAdd operation = coalesce(db.operation.name, code.function)
| filter isNotNull(operation)
| makeTimeseries ops = count(), by:{operation}, interval:5m
```

3. DynamoDB Monitoring

Amazon DynamoDB is a fully managed key-value and document database. Monitoring focuses on operation latency, consumed capacity, and identifying hot partitions through query patterns.

```
// DynamoDB operation performance by table
fetch spans, from:-1h
| filter db.system == "dynamodb"
| filter isNotNull(db.operation.name)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    errors = countIf(span.status_code == "error")
}, by:{db.namespace, db.operation.name}
| sort call_count desc
```

```
// DynamoDB Scan vs Query ratio – Scans are expensive and should be minimized
fetch spans, from:-1h
| filter db.system == "dynamodb"
| filter in(db.operation.name, {"Query", "Scan"})
| summarize {
    op_count = count(),
    avg_ms = avg(duration) / 1ms
}, by:{db.operation.name}
| sort op_count desc
```

Warning: DynamoDB `scan` operations read every item in the table and consume significant read capacity. A high Scan-to-Query ratio indicates missing or underused indexes (Global Secondary Indexes). Aim for Query operations to dominate over Scans.

4. Cassandra Monitoring

Apache Cassandra uses CQL (Cassandra Query Language), which looks similar to SQL but has fundamentally different performance characteristics due to its distributed architecture.

```
// Cassandra operation breakdown by keyspace
fetch spans, from:-1h
| filter db.system == "cassandra"
| filter isNotNull(db.operation.name)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms
}, by:{db.namespace, db.operation.name}
| sort call_count desc
```

```
// Cassandra latency spikes – detect operations exceeding 200ms
fetch spans, from:-6h
| filter db.system == "cassandra"
| makeTimeseries total = count(),
                  slow = countIf(duration > 200ms),
                  interval:10m
```

5. Cosmos DB Monitoring

Azure Cosmos DB is a globally distributed multi-model database. Performance is measured in Request Units (RU), and monitoring focuses on operation latency, RU consumption patterns, and consistency-related behavior.

```
// Cosmos DB operation analysis
fetch spans, from:-1h
| filter db.system == "cosmosdb"
| filter isNotNull(db.operation.name)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    errors = countIf(span.status_code == "error")
}, by:{db.namespace, db.operation.name}
| sort call_count desc
```

6. Read vs Write Analysis

Understanding the read/write ratio is critical for NoSQL database tuning. Read-heavy workloads benefit from caching and read replicas, while write-heavy workloads may need partition optimization.

```
// Read vs Write ratio across all NoSQL databases
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
"couchbase"})
| filter isNotNull(db.operation.name)
| fieldsAdd rw_type = if(
    in(db.operation.name, {"find", "Query", "GetItem", "SELECT", "ReadItem",
"get", "search"}),
    then:"READ",
    else:"WRITE")
| summarize {
    op_count = count(),
    avg_ms = avg(duration) / 1ms
}, by:{db.system, rw_type}
| sort db.system asc, rw_type asc
```

```
// Read/Write ratio trend over time – detect shifting workload patterns
fetch spans, from:-6h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb"})
| filter isNotNull(db.operation.name)
| fieldsAdd rw_type = if(
  in(db.operation.name, {"find", "Query", "GetItem", "SELECT", "ReadItem",
    "get"}),
  then:"READ",
  else:"WRITE")
| makeTimeseries op_count = count(), by:{rw_type}, interval:15m
```

7. Cross-Database Comparison

When your environment uses multiple NoSQL databases, comparing their performance characteristics helps identify which systems need attention.

```
// Compare NoSQL database performance – volume, latency, and error rates
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
  "couchbase"})
| summarize {
  total_calls = count(),
  avg_ms = avg(duration) / 1ms,
  p95_ms = percentile(duration, 95) / 1ms,
  error_count = countIf(span.status_code == "error"),
  unique_namespaces = countDistinct(db.namespace)
}, by:{db.system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
  toDouble(total_calls)) * 100, decimals:2)
| sort total_calls desc
```

8. Summary and Next Steps

In this notebook you learned:

- How Dynatrace captures NoSQL database operations through span instrumentation
- MongoDB operation analysis by collection and performance profiling
- DynamoDB monitoring with Scan vs Query ratio analysis
- Cassandra latency analysis by keyspace and operation

- Cosmos DB operation monitoring
- Read vs Write ratio analysis for workload characterization
- Cross-database performance comparison techniques

Next Steps

- **DBMON-04: Cache and Messaging Monitoring** — Redis, Kafka, RabbitMQ, and Elasticsearch analysis
- **DBMON-05: Query Analysis** — Deep query analysis, N+1 detection, and optimization patterns

Where to Go Deeper

- **CLOUD series** (9 notebooks) — Cloud-DB integration for DynamoDB (CloudWatch RU/throttling), Cosmos DB (Azure native), Cloud SQL / Spanner / Bigtable / Firestore (GCP)
- **AIOPS series** — Davis anomaly detection on NoSQL throughput, error rate, and per-collection latency
- **DBMON-05** — Query analysis patterns including N+1 detection across NoSQL

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.